

Universidad Libre De Colombia

Facultad De Ingeniería

Proyecto 4

Sistema de Gestión de Mascotas para una Clínica Veterinaria
Arquitectura de microservicios con Java y Spring Boot

Presentado por:
Santiago Enrique Arquez Cervantes, Cristian David Diaz Alarcón,
Ginneth Natalia González Puentes, Doncel Castro Stephania

Docente:

Miguel Hernández Bejarano

Bogotá D.C

Introducción

Muchas clínicas veterinarias todavía manejan el registro de sus pacientes por medio de hojas de cálculo, cuadernos o archivos físicos. Esta forma de trabajar genera problemas

frecuentes: se pierde información, los datos se duplican, programar citas se vuelve complicado y es difícil hacer seguimiento a los tratamientos de cada mascota.

Para atender esta necesidad, el presente trabajo desarrolla una plataforma distribuida que centraliza la información de propietarios, mascotas y citas veterinarias. La solución se construyó con microservicios en Java y Spring Boot, que exponen APIs REST y se comunican entre sí. Los datos se guardan mediante JPA, la documentación de los servicios se genera con Swagger/OpenAPI, las pruebas se realizaron con JUnit 5 y Mockito, y el despliegue se hace con Docker Compose.

En el presente documento se evidencia el diseño de la arquitectura, la implementación de cada servicio, las pruebas realizadas y el proceso de despliegue.

Objetivo general

Diseñar e implementar una solución para una clínica veterinaria basada en microservicios utilizando Java y Spring Boot para gestionar propietarios, mascotas y citas médicas veterinarias mediante APIs REST e integración entre servicios.

Microservicios del Proyecto

Servicio	Puerto	Base de datos	Responsabilidad
Owner-Service	8081	mysql-owners (owners_db)	Gestiona la información de los propietarios.
Pet-Service	8082	mysql-pets (pets_db)	Administra la información clínica básica de las mascotas.
Appointment-Service	8083	mysql-appointments (appointments_db)	Gestiona la programación y control de citas veterinarias.

Requerimientos Funcionales

El sistema debe cumplir con los siguientes requerimientos funcionales:

Owner-Service (Fase 1):

RF_01. El sistema debe permitir la gestión de propietarios con la administración de un CRUD completo (id, nombre, apellido, documento, telefono, correo, direccion, fechaRegistro) mediante GET/owners, GET /owners/{id}, POST /owners, PUT /owners/{id} y DELETE /owners/{id}.

RF_02. El Sistema no debe permitir documentos duplicados.

RF_03. El sistema no debe permitir correos electrónicos repetidos.

RF_04. El sistema debe exigir un teléfono de contacto para todos los propietarios.

RF_05. El sistema no debe permitir la eliminación de propietarios con mascotas asociadas (consulta a Pet-Service).

Pet-Service (Fase 2):

RF_06. El sistema debe permitir la gestión de mascotas con la administración de un CRUD desarrollado por completo (id, nombre, especie, raza, sexo, fechaNacimiento, peso, color, ownerId, estado) y consultar las mascotas de un propietario con GET /pets/owner/{ownerId}.

RF_07. El sistema debe permitir el manejo de los estados de la mascota: Activo, En Tratamiento, Recuperado y Fallecido.

RF_08. Antes de registrar una mascota, el sistema debe consultar a Owner-Service para verificar que el propietario exista; toda mascota debe estar asociada a un propietario válido.

RF_09. El sistema no debe permitir mascotas sin nombre y registrar correctamente la especie y la raza.

RF_10. El sistema no debe permitir fechas de nacimiento futuras.

Appointment-Service (Fase 3):

RF_11. El sistema debe permitir la gestión de citas por medio de un CRUD completo (id, petId, fecha, hora, motivoConsulta, observaciones, estado) y consultar las citas de una mascota con GET /appointments/pet/{petId}.

RF_12. El sistema debe permitir el manejo de los estados de la cita: Programada, Confirmada, Atendida y Cancelada.

RF_13. El sistema no debe programar citas para mascotas inexistentes.

RF_14. El sistema no debe permitir citas en fechas pasadas.

RF_15. El sistema debe evitar citas duplicadas en la misma fecha y hora.

RF_16. El sistema no debe permitir modificaciones de citas ya atendidas.

Integración, pruebas y despliegue (Fases 4 y 5):

RF_17. El sistema debe permitir la consulta de una cita completa (cita + mascota + propietario) mediante la cadena Appointment-Service → Pet-Service → Owner-Service.

RF_18. El sistema debe permitir la comunicación de los servicios con RestTemplate, usando DTOs de integración, timeouts de conexión y lectura configurables.

RF_19. El sistema debe manejar adecuadamente la indisponibilidad de servicios (HTTP 503 con mensaje claro) y devolver errores con un formato uniforme.

RF_20. El sistema debe documentar las APIs con Swagger/OpenAPI y entregar una colección Postman.

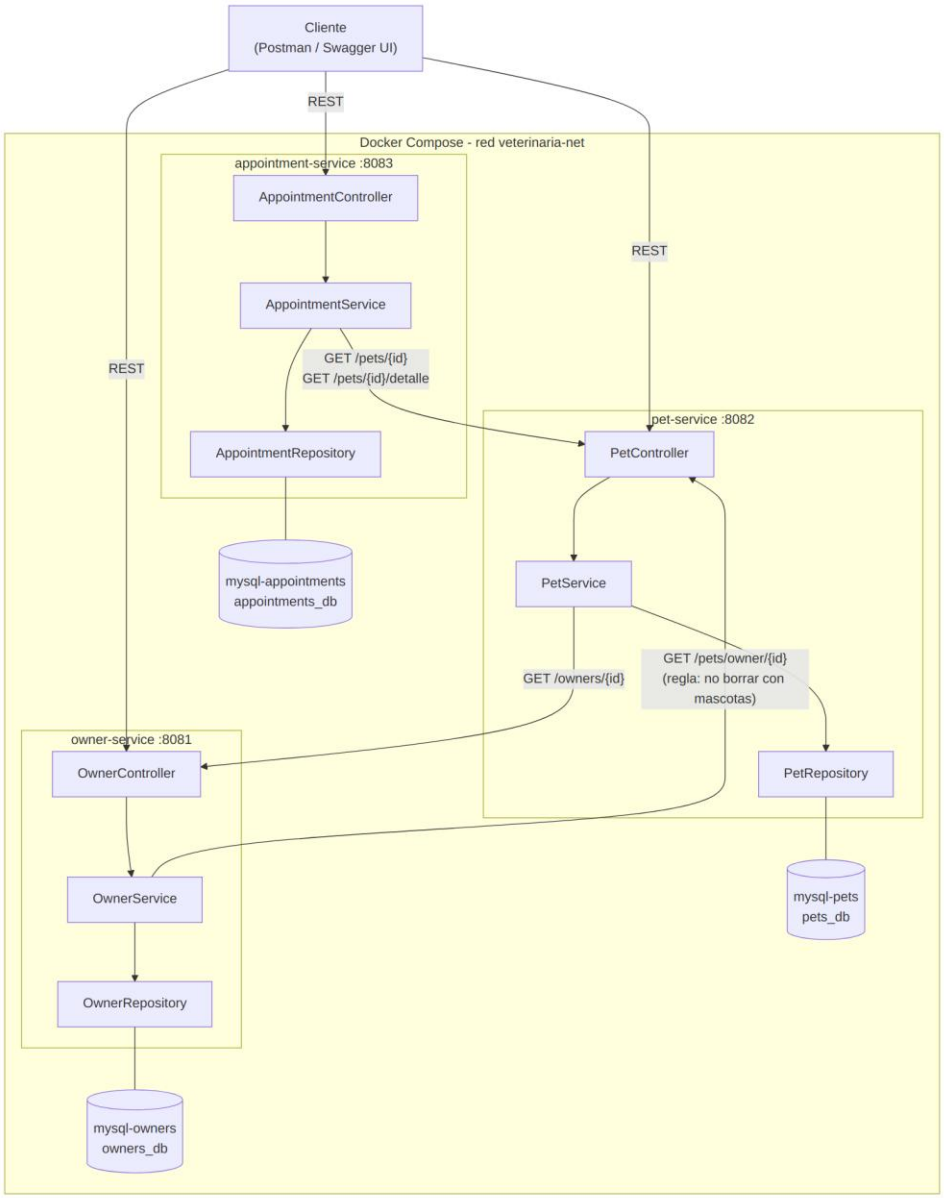
RF_21. El sistema debe contar con pruebas unitarias y de integración para los tres servicios.

RF_22. El sistema debe desplegar de manera correcta los tres microservicios y sus tres bases de datos MySQL con Docker Compose.

Arquitectura General

Cada microservicio es independiente, tiene su propia base de datos y se organiza por capas (controlador, servicio, repositorio y cliente REST), con un archivo por cada capa. No existen claves foráneas entre servicios: Pet.ownerId y Appointment.petId son referencias lógicas que se validan mediante llamadas REST. Los tres servicios y sus bases de datos se conectan en la red veterinaria-net de Docker Compose.

Diagrama de arquitectura (Mermaid):



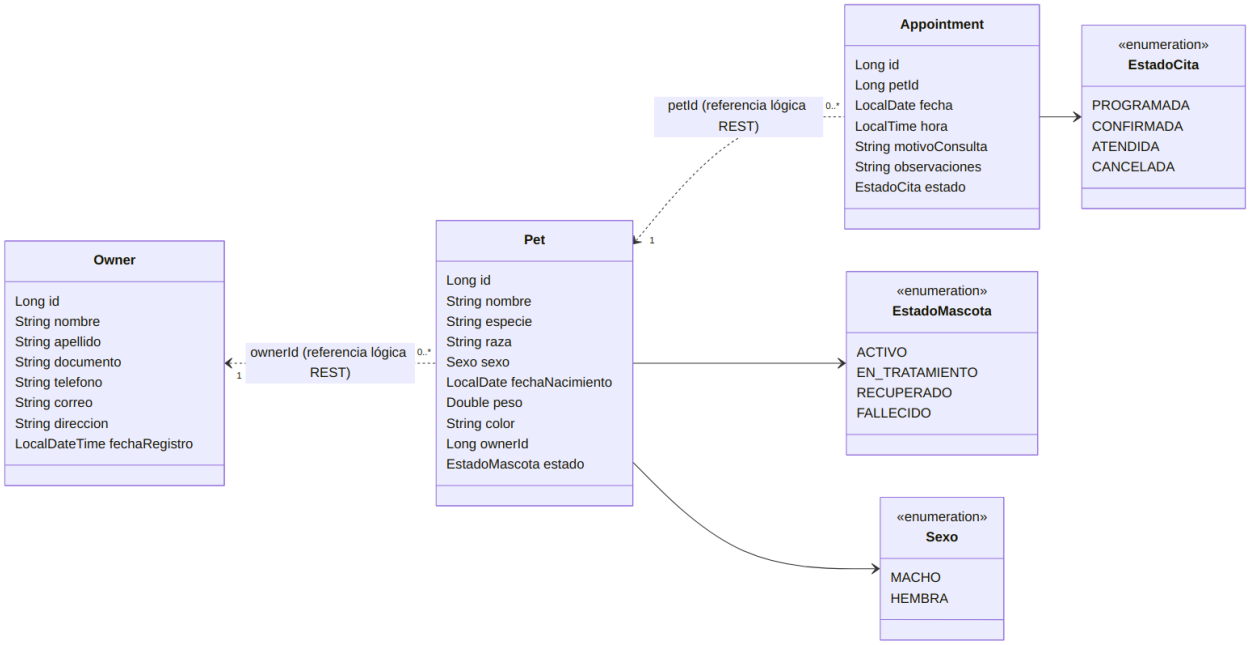
Código Mermaid del diagrama:

```
flowchart TB
    Cliente["Cliente<br/>(Postman / Swagger UI)"]

    subgraph Docker["Docker Compose - red veterinaria-net"]
        direction TB
        subgraph A["appointment-service :8083"]
            AC["AppointmentController"] --> AS["AppointmentService"] --> AR["AppointmentRepository"]
        end
        subgraph P["pet-service :8082"]
            PC["PetController"] --> PS["PetService"] --> PR["PetRepository"]
        end
        subgraph O["owner-service :8081"]
            OC["OwnerController"] --> OS["OwnerService"] --> OR["OwnerRepository"]
        end
        DBA[("mysql-appointments<br/>appointments_db")]
        DBP[("mysql-pets<br/>pets_db")]
        DBO[("mysql-owners<br/>owners_db")]
    end

    Cliente -->|REST| AC
    Cliente -->|REST| PC
    Cliente -->|REST| OC
    AS -->|"GET /pets/{id}<br/>GET /pets/{id}/detalle"| PC
    PS -->|"GET /owners/{id}"| OC
    OS -->|"GET /pets/owner/{id}<br/>(regla: no borrar con mascotas)"| PC
    AR --- DBA
    PR --- DBP
    OR --- DBO
```

Diagrama de clases



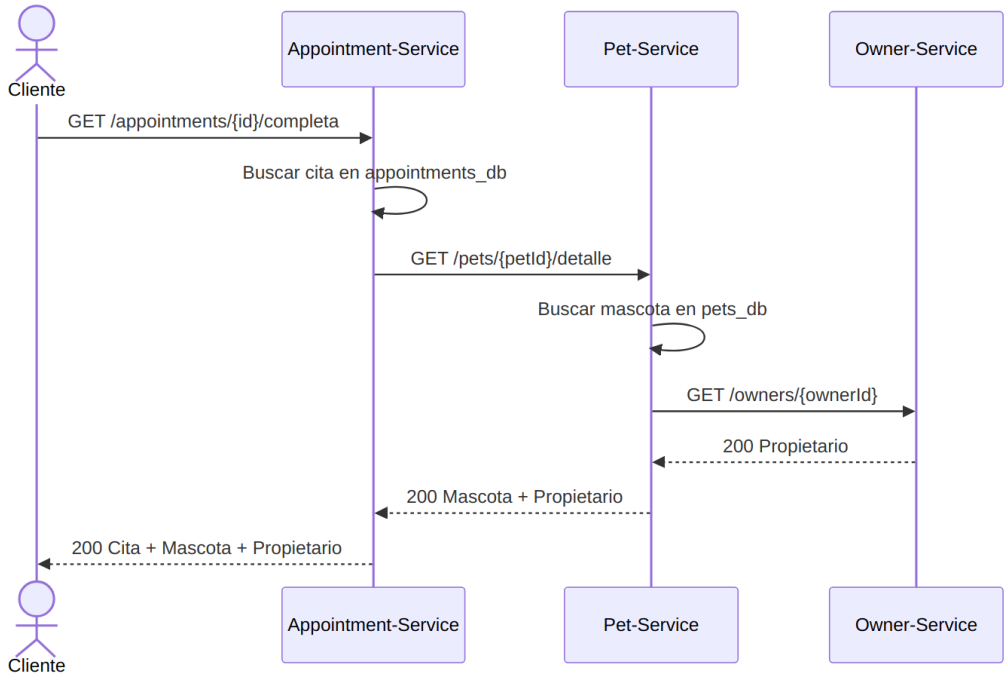
Código diagrama de clases:

```
classDiagram
    direction LR
    class Owner {
        Long id
        String nombre
        String apellido
        String documento
        String telefono
        String correo
        String direccion
        LocalDateTime fechaRegistro
    }
    class Mascota {
        Long id
        String nombre
        String especie
        String raza
        Sexo sexo
        LocalDate fechaNacimiento
        Double peso
        String color
        Long ownerId
        EstadoMascota estado
    }
    class Cita{
        Long id
        Long petId
        LocalDate fecha
        LocalTime hora
        String motivoConsulta
        String observaciones
        EstadoCita estado
    }
    class EstadoMascota {
        <<enumeration>>
        ACTIVO
        EN_TRATAMIENTO
        RECUPERADO
        FALLECIDO
```

```
}
class EstadoCita {
    <<enumeration>>
    PROGRAMADA
    CONFIRMADA
    ATENDIDA
    CANCELADA
}
class Sexo {
    <<enumeration>>
    MACHO
    HEMBRA
}

Owner "1" <.. "0..*" Pet : ownerId (referencia lógica REST)
Pet "1" <.. "0..*" Appointment : petId (referencia lógica REST)
Pet --> EstadoMascota
Pet --> Sexo
Appointment --> EstadoCita
```

Diagrama de secuencia: consultar cita completa


Cliente
Appointment-Service
Pet-Service
Owner-Service
GET /appointments/{id}/completa
Buscar cita en appointments_db
GET /pets/{petId}/detalle
Buscar mascota en pets_db
GET /owners/{ownerId}
200 Propietario
200 Mascota + Propietario
200 Cita + Mascota + Propietario
Cliente
Appointment-Service
Pet-Service
Owner-Service

Diseño de las APIs REST

Owner-Service (puerto 8081):

Método	Ruta	Descripción / códigos
GET	/owners	Lista propietarios (200)
GET	/owners/{id}	Obtiene un propietario (200 / 404)
POST	/owners	Registra un propietario (201 / 400 / 409)
PUT	/owners/{id}	Actualiza (200 / 400 / 404 / 409)
DELETE	/owners/{id}	Elimina si no tiene mascotas (204 / 404 / 409 / 503)

Pet-Service (puerto 8082):

Método	Ruta	Descripción / códigos
GET	/pets	Lista mascotas (200)
GET	/pets/{id}	Obtiene una mascota (200 / 404)
GET	/pets/owner/{ownerId}	Mascotas de un propietario (200)
GET	/pets/{id}/detalle	Mascota + propietario; consulta a Owner-Service (200 / 404 / 503)
POST	/pets	Registra mascota validando propietario (201 / 400 / 503)
PUT	/pets/{id}	Actualiza (200 / 400 / 404)
DELETE	/pets/{id}	Elimina (204 / 404)

Appointment-Service (puerto 8083):

Método	Ruta	Descripción / códigos
GET	/appointments	Lista citas (200)
GET	/appointments/{id}	Obtiene una cita (200 / 404)
GET	/appointments/pet/{petId}	Citas de una mascota (200)
GET	/appointments/{id}/completa	Cita + mascota + propietario (200 / 404 / 503)
POST	/appointments	Programa una cita (201 / 400 / 404 / 409 / 503)
PUT	/appointments/{id}	Actualiza (200 / 404 / 409)
DELETE	/appointments/{id}	Elimina si no está atendida (204 / 404 / 409)

Código Java

A continuación, se presenta el código fuente completo y comentado de cada microservicio. Todos siguen la misma organización de archivos: la clase principal (que también configura el RestTemplate), la entidad con sus enumerados, el repositorio, Dtos (todos los objetos JSON de entrada y salida), Errores (excepciones de negocio y su manejador de errores HTTP), el cliente hacia otro servicio, el servicio con las reglas de negocio y el controlador REST.

https://github.com/SantiagoArquez/Veterianrias.

Docker Compose

docker-compose.yml

```
services:

  # ---------------- Bases de datos (una por microservicio) ----------------
  mysql-owners:
    image: mysql:8.4
    container_name: mysql-owners
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD:-root}
      MYSQL_DATABASE: owners_db
    ports:
      - "3307:3306"
    volumes:
      - owners-data:/var/lib/mysql
    healthcheck:
      test: ["CMD-SHELL", "mysqladmin ping -h localhost -uroot -p$$MYSQL_ROOT_PASSWORD --silent"]
      interval: 10s
      timeout: 5s
      retries: 10
    networks:
      - veterinaria-net

  mysql-pets:
    image: mysql:8.4
    container_name: mysql-pets
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD:-root}
      MYSQL_DATABASE: pets_db
    ports:
      - "3308:3306"
    volumes:
      - pets-data:/var/lib/mysql
    healthcheck:
      test: ["CMD-SHELL", "mysqladmin ping -h localhost -uroot -p$$MYSQL_ROOT_PASSWORD --silent"]
      interval: 10s
      timeout: 5s
      retries: 10
    networks:
      - veterinaria-net

  mysql-appointments:
    image: mysql:8.4
```

```
    container_name: mysql-appointments
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD:-root}
      MYSQL_DATABASE: appointments_db
    ports:
      - "3309:3306"
    volumes:
      - appointments-data:/var/lib/mysql
    healthcheck:
      test: ["CMD-SHELL", "mysqladmin ping -h localhost -uroot -p$$MYSQL_ROOT_PASSWORD --silent"]
      interval: 10s
      timeout: 5s
      retries: 10
    networks:
      - veterinaria-net

  # ---------------- Microservicios ----------------
  owner-service:
    build: ./owner-service
    container_name: owner-service
    depends_on:
      mysql-owners:
        condition: service_healthy
    environment:
      DB_URL: jdbc:mysql://mysql-
owners:3306/owners_db?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC
      DB_USER: root
      DB_PASSWORD: ${MYSQL_ROOT_PASSWORD:-root}
      PET_SERVICE_URL: http://pet-service:8082
    ports:
      - "8081:8081"
    restart: on-failure
    networks:
      - veterinaria-net

  pet-service:
    build: ./pet-service
    container_name: pet-service
    depends_on:
      mysql-pets:
        condition: service_healthy
    environment:
      DB_URL: jdbc:mysql://mysql-
pets:3306/pets_db?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC
      DB_USER: root
      DB_PASSWORD: ${MYSQL_ROOT_PASSWORD:-root}
      OWNER_SERVICE_URL: http://owner-service:8081
    ports:
      - "8082:8082"
    restart: on-failure
    networks:
      - veterinaria-net

  appointment-service:
    build: ./appointment-service
    container_name: appointment-service
    depends_on:
      mysql-appointments:
        condition: service_healthy
    environment:
      DB_URL: jdbc:mysql://mysql-
appointments:3306/appointments_db?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC
      DB_USER: root
      DB_PASSWORD: ${MYSQL_ROOT_PASSWORD:-root}
      PET_SERVICE_URL: http://pet-service:8082
    ports:
      - "8083:8083"
    restart: on-failure
    networks:
      - veterinaria-net
```

```
networks:
  veterinaria-net:
    driver: bridge

volumes:
  owners-data:
  pets-data:
  appointments-data:
```

Plan de pruebas

Pruebas unitarias (JUnit 5 + Mockito):

Servicio	Casos cubiertos
Owner-Service	Registro de propietarios; documento y correo duplicados; teléfono obligatorio; consulta por identificador (existente e inexistente); actualización de datos; eliminación sin mascotas y bloqueo con mascotas asociadas.
Pet-Service	Registro con propietario válido (estado inicial Activo); propietario inexistente; Owner-Service no disponible; nombre vacío; fecha de nacimiento futura; consulta inexistente; detalle mascota + propietario; actualización de estado.
Appointment-Service	Creación de citas; mascota inexistente; fecha pasada; misma fecha y hora; actualización de estado; cita atendida no modificable ni eliminable; horario ocupado al reprogramar; consulta completa cita + mascota + propietario.

Pruebas de integración

Cada servicio incluye una clase de pruebas de integración (…ApiTest) con @SpringBootTest, MockMvc y base H2 en memoria; los clientes REST hacia otros servicios se sustituyen con @MockBean. Los cinco casos del enunciado se cubren así:

Caso	Descripción	Resultado esperado	Prueba
1	Registrar propietario: POST /owners	HTTP 201 CREATED	OwnerApiTest.postOwners_devuelve201
2	Registrar mascota asociada (Pet → Owner)	Propietario validado y mascota registrada (201)	PetApiTest.postPets_conPropietarioValido_devuelve201
3	Programar cita (Appointment → Pet)	Mascota validada y cita creada (201)	AppointmentApiTest.postAppointments_conMascotaValida_devuelve201
4	Programar cita para mascota inexistente	HTTP 404 NOT FOUND	AppointmentApiTest.postAppointments_mascotaInexistente_devuelve404

Caso	Descripción	Resultado esperado	Prueba
5	Crear mascota para propietario inexistente	HTTP 400 BAD REQUEST	PetApiTest.postPets_conPropietarioInexistente_devuelve400

Además, la colección Postman (postman/Clinica_Veterinaria.postman_collection.json) automatiza estos mismos casos y las reglas de negocio contra los servicios desplegados con Docker Compose.

Resultados de la ejecución

Las pruebas se ejecutaron con mvn test en cada servicio el 4 de noviembre de 2026 (GitHub Codespaces, Java 25.0.5, Spring Boot 3.3.5). En total se ejecutaron 49 pruebas, todas exitosas.

Servicio	Unitarias	Integración	Total	Resultado
Owner-Service	10	8	18	BUILD SUCCESS
Pet-Service	8	7	15	BUILD SUCCESS
Appointment-Service	9	7	16	BUILD SUCCESS
Total	27	22	49	0 fallos, 0 errores

Observación técnica: el entorno de ejecución tiene instalado Java 25, mientras que el proyecto compila para Java 21 (release 21). La versión de Byte Buddy que usa Mockito en Spring Boot 3.3.5 solo reconoce oficialmente hasta Java 23, por lo que se agregó en cada pom.xml la propiedad argLine con -Dnet.bytebuddy.experimental=true. Este ajuste solo afecta a la ejecución de las pruebas y no a la aplicación; en Docker los servicios se ejecutan sobre una imagen de Java 21.

Pantallas de Ejecución

Las siguientes evidencias corresponden a la ejecución real del sistema el 4 de noviembre de 2026 en un entorno GitHub Codespaces (Java 25.0.5, Spring Boot 3.3.5, Docker Compose). Se presenta la salida de consola de cada comando; los comandos aparecen resaltados en azul con el símbolo $ y la respuesta del sistema en gris.

1. Arranque de los tres microservicios:

Comando: docker compose up --build. Los tres servicios inician y quedan escuchando en los puertos 8081, 8082 y 8083, cada uno conectado a su propia base de datos MySQL (extracto de los logs).

```
$ docker compose up --build
owner-service        | 2026-11-04T03:37:46.567Z  INFO 1 --- [owner-service] [           main]
o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat started on port 8081 (http) with context path '/'
owner-service        | 2026-11-04T03:37:46.670Z  INFO 1 --- [owner-service] [           main]
c.clinica.owner.OwnerServiceApplication  : Started OwnerServiceApplication in 34.466 seconds
(process running for 38.038)
appointment-service  | 2026-11-04T03:37:46.991Z  INFO 1 --- [appointment-service] [           main]
o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat started on port 8083 (http) with context path '/'
appointment-service  | 2026-11-04T03:37:47.066Z  INFO 1 --- [appointment-service] [           main]
c.c.a.AppointmentServiceApplication      : Started AppointmentServiceApplication in 34.787 seconds
(process running for 38.123)
pet-service          | 2026-11-04T03:37:47.248Z  INFO 1 --- [pet-service] [           main]
o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat started on port 8082 (http) with context path '/'
pet-service          | 2026-11-04T03:37:47.292Z  INFO 1 --- [pet-service] [           main]
com.clinica.pet.PetServiceApplication    : Started PetServiceApplication in 34.393 seconds (process
running for 37.94)
```

2. Caso 1: registrar propietario → 201 Created:

POST /owners en Owner-Service.

```
$ curl -i -X POST localhost:8081/owners -H "Content-Type: application/json" \
    -d
'{"nombre":"Ana","apellido":"Pérez","documento":"1001","telefono":"3001234567","correo":"ana@mail.co
m","direccion":"Calle 1"}'
HTTP/1.1 201
Location: http://localhost:8081/owners/1
Content-Type: application/json
Transfer-Encoding: chunked
Date: Wed, 04 Nov 2026 03:49:59 GMT

{"id":1,"nombre":"Ana","apellido":"Pérez","documento":"1001","telefono":"3001234567","correo":"ana@m
ail.com","direccion":"Calle 1","fechaRegistro":"2026-11-04T03:49:59.051985521"}
```

3. Caso 2: registrar mascota asociada a un propietario → 201 Created:

POST /pets en Pet-Service. Pet-Service consulta a Owner-Service y verifica que el propietario 1 exista; la mascota queda en estado ACTIVO.

```
$ curl -i -X POST localhost:8082/pets -H "Content-Type: application/json" \
    -d
'{"nombre":"Firulais","especie":"Perro","raza":"Labrador","sexo":"MACHO","fechaNacimiento":"2020-05-
01","peso":25.5,"color":"Negro","ownerId":1}'
HTTP/1.1 201
Location: http://localhost:8082/pets/1
Content-Type: application/json
Transfer-Encoding: chunked
```

```
Date: Wed, 04 Nov 2026 03:50:19 GMT


{"id":1,"nombre":"Firulais","especie":"Perro","raza":"Labrador","sexo":"MACHO","fechaNacimiento":"20
20-05-01","peso":25.5,"color":"Negro","ownerId":1,"estado":"ACTIVO"}
```

4. Caso 3: programar una cita → 201 Created:

POST /appointments en Appointment-Service. Appointment-Service consulta a Pet-Service y verifica que la mascota 1 exista; la cita queda PROGRAMADA.

```
$ curl -i -X POST localhost:8083/appointments -H "Content-Type: application/json" \
    -d '{"petId":1,"fecha":"2030-01-15","hora":"10:30","motivoConsulta":"Vacunación"}'
HTTP/1.1 201
Location: http://localhost:8083/appointments/1
Content-Type: application/json
Transfer-Encoding: chunked
Date: Wed, 04 Nov 2026 03:50:30 GMT


{"id":1,"petId":1,"fecha":"2030-01-
15","hora":"10:30:00","motivoConsulta":"Vacunación","observaciones":null,"estado":"PROGRAMADA"}
```

5. Caso 4: cita para una mascota inexistente → 404 Not Found:

La mascota 999 no existe en Pet-Service.

```
$ curl -i -X POST localhost:8083/appointments -H "Content-Type: application/json" \
    -d '{"petId":999,"fecha":"2030-01-15","hora":"11:00","motivoConsulta":"Control"}'
HTTP/1.1 404
Content-Type: application/json
Transfer-Encoding: chunked
Date: Wed, 04 Nov 2026 03:50:40 GMT


{"timestamp":"2026-11-04T03:50:40.353044872","status":404,"error":"Not Found","mensaje":"La mascota
con id 999 no existe","ruta":"/appointments"}
```

6. Caso 5: mascota para un propietario inexistente → 400 Bad Request:

El propietario 999 no existe en Owner-Service.

```
$ curl -i -X POST localhost:8082/pets -H "Content-Type: application/json" \
    -d '{"nombre":"Rex","especie":"Perro","raza":"Criollo","sexo":"MACHO","fechaNacimiento":"2021-
01-01","peso":10,"ownerId":999}'
HTTP/1.1 400
Content-Type: application/json
Transfer-Encoding: chunked
Date: Wed, 04 Nov 2026 03:51:10 GMT
Connection: close


{"timestamp":"2026-11-04T03:51:10.825617424","status":400,"error":"Bad Request","mensaje":"El
propietario con id 999 no existe","ruta":"/pets"}
```

7. Consulta de cita completa → 200 OK:

GET /appointments/1/completa. Se recorre toda la cadena Appointment-Service → Pet-Service → Owner-Service y se devuelven la cita, la mascota y el propietario en una sola respuesta.

```
$ curl -i localhost:8083/appointments/1/completa
HTTP/1.1 200
Content-Type: application/json
Transfer-Encoding: chunked
Date: Wed, 04 Nov 2026 03:51:18 GMT
```

```
{"cita":{"id":1,"petId":1,"fecha":"2030-01-
15","hora":"10:30:00","motivoConsulta":"Vacunación","observaciones":null,"estado":"PROGRAMADA"},"mas
cota":{"id":1,"nombre":"Firulais","especie":"Perro","raza":"Labrador","sexo":"MACHO","fechaNacimient
o":"2020-05-
01","peso":25.5,"color":"Negro","ownerId":1,"estado":"ACTIVO"},"propietario":{"id":1,"nombre":"Ana",
"apellido":"Pérez","documento":"1001","telefono":"3001234567","correo":"ana@mail.com","direccion":"C
alle 1","fechaRegistro":"2026-11-04T03:49:59.051986"}}
```


8. Regla de negocio: no eliminar un propietario con mascotas → 409 Conflict:

DELETE /owners/1. Owner-Service consulta a Pet-Service y encuentra mascotas asociadas.

```
$ curl -i -X DELETE localhost:8081/owners/1
HTTP/1.1 409
Content-Type: application/json
Transfer-Encoding: chunked
Date: Wed, 04 Nov 2026 03:51:48 GMT

{"timestamp":"2026-11-04T03:51:48.020576403","status":409,"error":"Conflict","mensaje":"No se puede
eliminar el propietario 1 porque tiene mascotas asociadas","ruta":"/owners/1"}
```

9. Regla de negocio: no duplicar fecha y hora de una cita → 409 Conflict:

Se intenta programar otra cita el 2030-01-15 a las 10:30, horario ya ocupado por la cita 1.

```
$ curl -i -X POST localhost:8083/appointments -H "Content-Type: application/json" \
    -d '{"petId":1,"fecha":"2030-01-15","hora":"10:30","motivoConsulta":"Otra"}'
HTTP/1.1 409
Content-Type: application/json
Transfer-Encoding: chunked
Date: Wed, 04 Nov 2026 03:51:57 GMT

{"timestamp":"2026-11-04T03:51:57.888371824","status":409,"error":"Conflict","mensaje":"Ya existe
una cita programada para el 2030-01-15 a las 10:30","ruta":"/appointments"}
```

10. Servicio no disponible → 503 Service Unavailable:

Se detiene Owner-Service y se intenta registrar una mascota. Pet-Service no puede validar al propietario y responde con un error controlado; luego se reinicia Owner-Service.

```
$ docker compose stop owner-service
 ✓ Container owner-service Stopped                                    0.4s
```

```
$ curl -i -X POST localhost:8082/pets -H "Content-Type: application/json" \
    -d '{"nombre":"Luna","especie":"Gato","raza":"Siamés","sexo":"HEMBRA","fechaNacimiento":"2022-
01-01","peso":4,"ownerId":1}'
HTTP/1.1 503
Content-Type: application/json
Transfer-Encoding: chunked
Date: Wed, 04 Nov 2026 03:53:19 GMT
Connection: close

{"timestamp":"2026-11-04T03:53:19.251790996","status":503,"error":"Service
Unavailable","mensaje":"Owner-Service no está disponible en este momento","ruta":"/pets"}
```

```
$ docker compose start owner-service
 ✓ Container mysql-owners  Healthy                                    0.5s
 ✓ Container owner-service Started                                    0.2s
```

11. Pruebas automáticas de Owner-Service → BUILD SUCCESS:

Comando: mvn test (extracto de la salida). 8 pruebas de integración y 10 unitarias.

```
$ cd owner-service && mvn test
[INFO] Running com.clinica.owner.OwnerApiTest
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 10.75 s -- in
com.clinica.owner.OwnerApiTest
[INFO] Running com.clinica.owner.OwnerServiceTest
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.512 s -- in
com.clinica.owner.OwnerServiceTest
[INFO] Results:
[INFO] Tests run: 18, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time:  16.365 s
[INFO] Finished at: 2026-11-04T03:58:34Z
```

12. Pruebas automáticas de Pet-Service → BUILD SUCCESS:

Comando: mvn test (extracto de la salida). 7 pruebas de integración y 8 unitarias.

```
$ cd pet-service && mvn test
[INFO] Running com.clinica.pet.PetServiceTest
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.637 s -- in
com.clinica.pet.PetServiceTest
[INFO] Running com.clinica.pet.PetApiTest
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 8.146 s -- in
com.clinica.pet.PetApiTest
[INFO] Results:
[INFO] Tests run: 15, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time:  21.436 s
[INFO] Finished at: 2026-11-04T03:59:50Z
```

13. Pruebas automáticas de Appointment-Service → BUILD SUCCESS:

Comando: mvn test (extracto de la salida). 7 pruebas de integración y 9 unitarias.

```
$ cd appointment-service && mvn test
[INFO] Running com.clinica.appointment.AppointmentApiTest
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 10.26 s -- in
com.clinica.appointment.AppointmentApiTest
[INFO] Running com.clinica.appointment.AppointmentServiceTest
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.388 s -- in
com.clinica.appointment.AppointmentServiceTest
[INFO] Results:
[INFO] Tests run: 16, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time:  20.456 s
[INFO] Finished at: 2026-11-04T04:00:36Z
```

Explicacion Tecnica de la Solucion

El sistema se compone de tres microservicios Spring Boot 3 (Java 21): Owner-Service, Pet-Service y Appointment-Service. Cada uno sigue una arquitectura por capas: el controlador REST recibe y valida la petición (anotaciones de Bean Validation sobre DTOs tipo record, agrupados en Dtos.java), la capa de servicio aplica las reglas de negocio, el repositorio Spring Data JPA persiste en su propia base MySQL y los clientes REST se encargan de hablar con otros servicios. Las entidades nunca se exponen directamente: se usan DTOs de entrada (Request) y salida (Response).

La integración se hace con RestTemplate configurado con timeouts de conexión (2 s) y de lectura (5 s). Pet-Service consulta GET /owners/{id} antes de registrar o reasignar una mascota; Appointment-Service consulta GET /pets/{id} antes de crear una cita. Para la consulta completa, Appointment-Service llama a GET /pets/{id}/detalle, y Pet-Service a su vez completa la respuesta con los datos del propietario, reproduciendo la cadena Appointment → Pet → Owner. Como los datos viven en bases distintas, la relación se mantiene mediante referencias lógicas (ownerId, petId) y no con claves foráneas. La regla de no eliminar propietarios con mascotas exige además que Owner-Service consulte GET /pets/owner/{ownerId}.

Cada cliente REST traduce los fallos remotos: un 404 se convierte en la excepción NoEncontrado (que según el caso termina como 404 o, en el registro de mascotas, como 400) y cualquier otro fallo de comunicación se convierte en ServicioNoDisponible, que el Manejador de Errores.java responde como 503 con un cuerpo uniforme (timestamp, status, error, mensaje, ruta). Los conflictos de negocio (duplicados, cita atendida) responden 409.

Para el despliegue, cada servicio tiene un Dockerfile multietapa (Maven para compilar y una imagen JRE para ejecutar). El docker-compose.yml levanta tres contenedores MySQL con healthcheck, los tres microservicios que esperan a que su base esté saludable, una red común (veterinaria-net), volúmenes persistentes y variables de entorno para credenciales

y URLs entre servicios. Las pruebas unitarias usan Mockito sobre la capa de servicio y las de integración usan MockMvc con H2, por lo que no dependen de Docker ni de MySQL.

Decisiones de diseño

- Las enumeraciones (estado de mascota, estado de cita, sexo) se guardan como texto y se escriben en mayúsculas (por ejemplo, EN_TRATAMIENTO).
- Una cita cancelada libera su horario: no cuenta en el control de citas duplicadas.
- Toda cita nueva queda en estado Programada; el estado solo cambia mediante PUT.
- Si Pet-Service no responde, Owner-Service rechaza el borrado de un propietario con 503, priorizando la integridad de los datos sobre la disponibilidad.

Conclusiones

Dividir el sistema en tres microservicios dejó claro qué hace cada uno: Owner-Service maneja los propietarios, Pet-Service las mascotas y Appointment-Service las citas, cada uno con su propia base de datos. Esto trajo un reto de diseño. Como los datos están en bases separadas, no hay claves foráneas, así que para saber si un propietario o una mascota existe hay que preguntárselo al otro servicio por REST. Las reglas de negocio también quedaron repartidas, y cada servicio se encarga de las que dependen de sus propios datos.

Comunicar servicios también trae problemas que en una aplicación única no existen. Por ejemplo, un servicio puede no responder. Por eso definimos tiempos de espera (timeouts) y convertimos los errores remotos en respuestas HTTP claras: 400, 404, 409 y 503. La regla de no eliminar propietarios que tienen mascotas muestra el costo de esta separación: Owner-Service ahora depende de Pet-Service para una sola operación. En el futuro se podría reducir usando eventos o guardando una copia local del número de mascotas.

Las pruebas confirmaron que el sistema se comporta como se esperaba. Los cinco casos del enunciado respondieron con los códigos previstos (201, 201, 201, 404 y 400). La consulta de una cita completa recorrió bien la cadena Appointment → Pet → Owner, las reglas de integridad devolvieron 409 y, al apagar Owner-Service, el sistema respondió con un 503 controlado. Además, las 49 pruebas automáticas (27 unitarias y 22 de integración) pasaron sin fallos.

Herramientas como Spring Boot, Spring Data JPA, Bean Validation, Swagger/OpenAPI y Docker Compose ahorraron mucho código repetitivo y permiten levantar todo el sistema con un solo comando. Como mejoras futuras se proponen un API Gateway, descubrimiento de servicios, autenticación con JWT, reintentos y circuit breaker en los clientes REST, y usar Flyway para las migraciones en lugar de `ddl-auto`.